

Guía DevOps Obras Públicas

Desde cero: qué se construyó, para qué sirve y cómo trabaja el equipo a partir de ahora

ROL	DevOps
CLOUD	Microsoft Azure
AMBIENTE ACTIVO	development - Chile Central
ESTADO	Infraestructura convergente y verificada

Documento interno del equipo - 01/09/2026

AUDITORÍA DE LAS CUATRO TARJETAS

Respuesta corta: ¿está todo hecho?

Sí, las cuatro tareas DevOps están realizadas.

Tres ya estaban completas. La mensajería era la única brecha: se implementó el 01/09/2026 con Azure Event Grid, se desplegó mediante Terraform y aceptó un evento técnico de prueba con HTTP 200.

Tarea	Estado comprobado	Evidencia
Configurar Resource Group y servicios	HECHO	Resource Groups por ambiente, Container Apps, ACR, Log Analytics, PostgreSQL y Event Grid en Azure.
Definir infraestructura como código	HECHO	Terraform administra recursos, permisos, secretos y estado remoto. Plan final: No changes.
Provisionar mensajería en Azure	HECHO	Tópico egt-obras-publicas-dev-events, CloudEvents 1.0, estado Succeeded y publicación técnica HTTP 200.
Armar pipelines	HECHO	CI, publicación de imágenes y deploy con plan/apply separados y autorización DevOps.

Importante: plataforma técnica completa no significa aplicación funcional completa.

El equipo de desarrollo todavía debe implementar el modelo de datos, migraciones y endpoints reales del dominio. Tampoco se crean suscripciones de eventos hasta que Core y los módulos consumidores confirmen los contratos.

CONCEPTOS DESDE CERO

1. Qué significa DevOps en este proyecto

DevOps conecta desarrollo y operaciones. El equipo escribe frontend y backend; DevOps prepara el lugar donde se ejecutan, automatiza la validación y controla cuándo una versión se despliega.

Cloud

Azure reemplaza al servidor configurado a mano. Los servicios existen como recursos administrados y reproducibles.

Infraestructura como código

Terraform describe la infraestructura en archivos versionados. Un plan muestra el cambio antes de aplicarlo.

CI

Integración continua: cada Pull Request ejecuta validaciones automáticas antes de fusionarse.

CD

Entrega/despliegue continuo: una versión validada se empaqueta y se despliega con autorización DevOps.

Contenedor

Una imagen Docker contiene la aplicación y sus dependencias para ejecutarse igual en todos los ambientes.

Ambiente

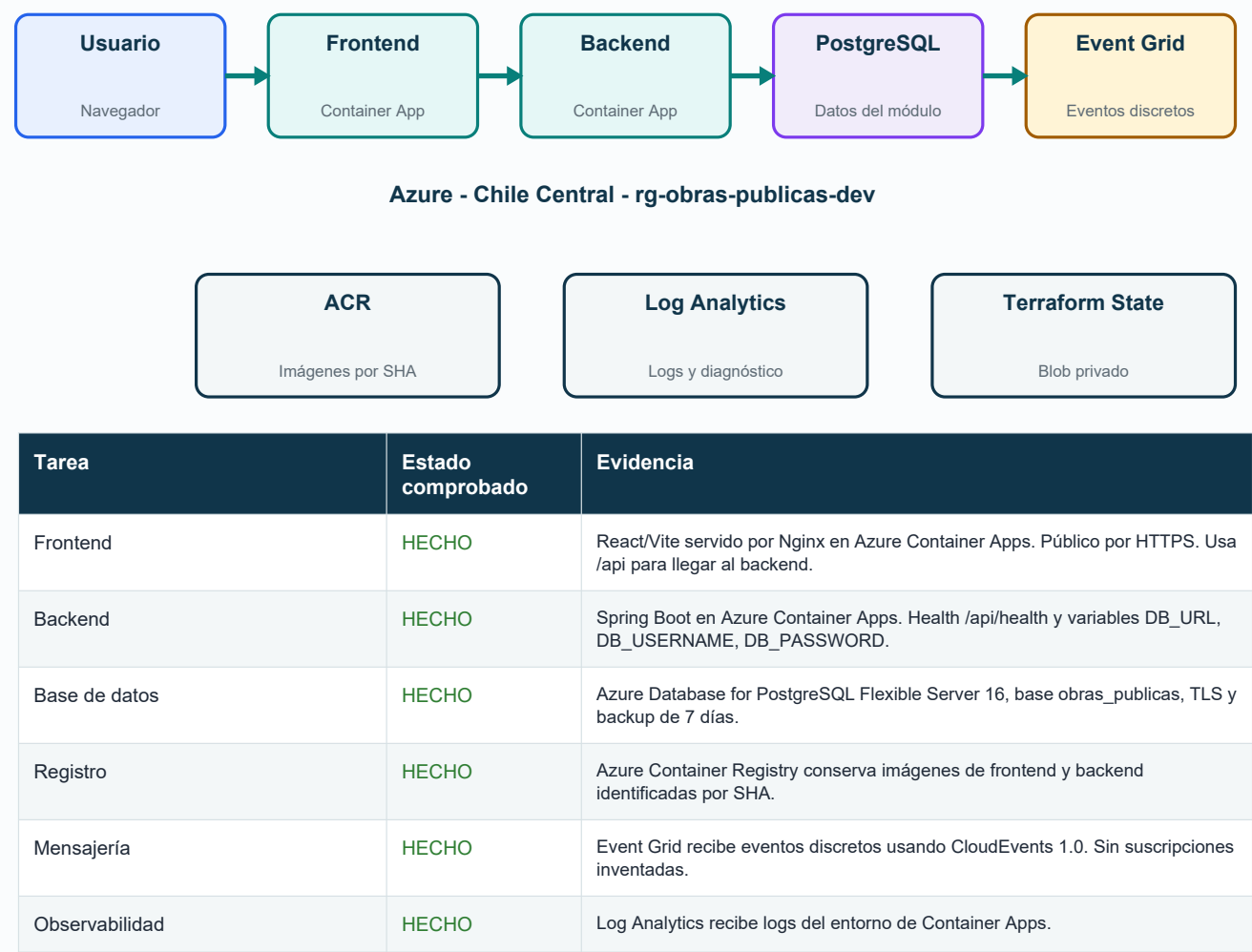
development sirve para integrar el sprint. test sirve para validar una release. No hay producción en este incremento.

Tu responsabilidad

Como DevOps, mantenés Azure, Terraform, pipelines, secretos, observabilidad, promoción y rollback. Además, autorizás los despliegues. El PO valida funcionalmente la release; no reemplaza tu aprobación operativa.

QUÉ CORRE Y CÓMO SE CONECTA

2. Arquitectura actual en Azure



QUÉ CREÓ TERRAFORM

3. Recursos y decisiones de costo

Tarea	Estado comprobado	Evidencia
rg-obras-publicas-dev	HECHO	Agrupar los recursos del ambiente development.
cae-obras-publicas-dev	HECHO	Entorno compartido donde corren los dos Container Apps.
ca-...-frontend / backend	HECHO	Contenedores con escalado de 0 a 1 réplica para reducir consumo académico.
psql-obras-publicas-dev-dfc86d	HECHO	PostgreSQL 16 B_Standard_B1ms, 32 GiB y retención mínima de backup.
egt-obras-publicas-dev-events	HECHO	Tópico Event Grid serverless para eventos discretos.
acrobaspública302ceba6	HECHO	ACR Basic compartido con imágenes por SHA.
log-obras-publicas-dev	HECHO	Workspace de logs con retención de 30 días.
Estado Terraform	HECHO	Blob privado, versionado y separado del código.

Por qué Event Grid

Los eventos del módulo representan cambios de estado. Event Grid es serverless y adecuado para routing reactivo. Service Bus se reevaluará si aparecen requisitos confirmados de orden estricto, transacciones o detección de duplicados.

No se deben crear recursos manualmente desde el portal. Si un cambio manual fuera inevitable, debe documentarse y reconciliarse inmediatamente en Terraform.

INFRAESTRUCTURA COMO CÓDIGO

4. Terraform: el plano de la infraestructura

Terraform compara lo escrito en Git con lo que existe en Azure. El resultado se llama plan. Solo después de revisarlo se autoriza el apply.

1. Código

Los archivos .tf describen recursos, dependencias, configuración y outputs no secretos.

2. Estado remoto

Terraform guarda IDs y valores sensibles en Azure Blob. El archivo de estado nunca va a Git.

3. Plan

Muestra altas, cambios y bajas. Una baja inesperada bloquea el despliegue hasta entenderla.

4. Apply

Ejecuta exactamente el plan aprobado sobre el mismo commit.

Lectura rápida de un plan

```
+ create      -> recurso nuevo
~ update      -> cambio sobre un recurso existente
- destroy      -> eliminación; requiere revisión especial
```

Plan de Event Grid: 1 to add, 1 to change, 0 to destroy

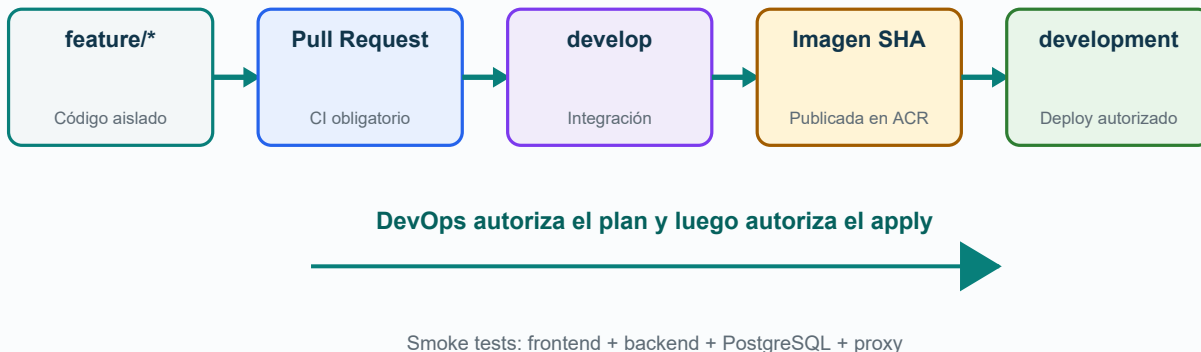
Plan posterior: No changes. Your infrastructure matches the configuration.

Regla de seguridad

Nunca subir terraform.tfstate, planes, tfvars reales, backend.hcl, claves, tokens ni contraseñas. Los cambios destructivos no se autorizan por rutina.

DEL CÓDIGO AL AMBIENTE DEVELOPMENT

5. Cómo funciona el pipeline



Pipeline de aplicación

- Instala dependencias con versiones bloqueadas.
- Ejecuta lint, pruebas y build.
- Construye una imagen Docker de frontend o backend.
- Publica la imagen en ACR usando el SHA del commit como versión inmutable.

Pipeline de infraestructura

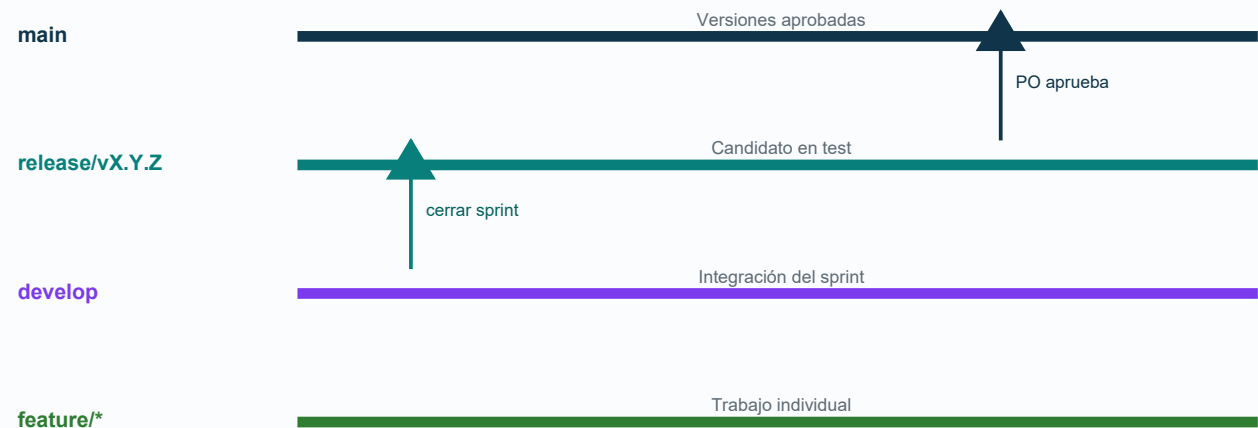
- Inicializa Terraform con estado remoto y autenticación OIDC.
- DevOps autoriza el job de plan.
- DevOps revisa el artefacto y autoriza el apply por separado.
- Después del deploy se ejecutan smoke tests y se confirma que no exista drift.

OIDC

GitHub obtiene una credencial temporal para Azure. No existe un client secret permanente guardado en GitHub.

RAMAS NO SON AMBIENTES

6. GitFlow y ambientes



Tarea	Estado comprobado	Evidencia
feature/*	HECHO	Nace desde develop. Trabajo aislado, pruebas locales y CI. No despliega automáticamente.
develop	HECHO	Integra las historias del sprint. Su versión validada puede desplegarse en development con autorización DevOps.
release/vX.Y.Z	PENDIENTE	Se crea al cerrar el sprint. Se desplegará en test cuando exista una release candidata.
main	HECHO	Conserva versiones aprobadas. Será la base de producción si ese ambiente se autoriza.
hotfix/*	PENDIENTE	Solo se usará cuando exista producción.

Clave

La rama indica qué estado tiene el código. El ambiente indica dónde se ejecuta una versión. develop no es una computadora ni un servidor.

QUÉ DEBEN HACER FRONTEND Y BACKEND

7. Flujo diario del equipo de desarrollo

Paso	Acción	Resultado esperado
1	Actualizar develop	Traer la última versión aprobada antes de comenzar.
2	Crear feature	Usar feature/nombre-claro desde develop.
3	Desarrollar y probar	Frontend: lint, test y build. Backend: Maven test/package. Migraciones: probarlas en PostgreSQL.
4	Abrir Pull Request	Destino develop. Explicar cambio, prueba, endpoints, variables y migraciones.
5	Corregir CI	No fusionar mientras una validación obligatoria falle.
6	Merge	Al fusionar, se construye una imagen identificable por SHA.
7	Deploy DevOps	DevOps revisa plan, autoriza apply y ejecuta smoke tests.
8	Validar en Azure	El equipo prueba la historia usando el ambiente development, no solo localhost.

Frontend

Debe usar rutas relativas como `/api/...` y nunca escribir `localhost` ni la URL concreta de Azure dentro del código.

Backend

Debe leer `DB_URL`, `DB_USERNAME` y `DB_PASSWORD`. Para Event Grid dispone de `EVENT_GRID_TOPIC_ENDPOINT` y `EVENT_GRID_ACCESS_KEY`.

EL FLUJO DE RELEASE

8. Cierre del sprint y ambiente test

El ambiente development cambia durante el sprint. test debe recibir una versión candidata identificable y estable.

1. Congelar alcance

Confirmar qué historias cumplen Definition of Done en develop.

2. Crear release

Crear release/vX.Y.Z desde el commit probado de develop.

3. Promover

Usar el mismo digest de imagen cuando el código no cambió.
No reconstruir sin motivo.

4. Autorizar test

DevOps revisa configuración y autoriza el despliegue al ambiente test.

5. Validar

PO ejecuta aceptación funcional. DevOps revisa logs, versión y rollback.

6. Cerrar

Fusionar la release en main, crear tag vX.Y.Z y devolver correcciones a develop.

Estado actual

El workflow de test está preparado, pero todavía no se desplegó una release. No existe ambiente de producción en el alcance actual.

Si se detecta un defecto en test, se corrige en la rama release y esa corrección vuelve a develop. hotfix se reserva para un futuro ambiente productivo.

TRES TEMAS QUE NO DEBEN MEZCLARSE

9. Datos, secretos y eventos

PostgreSQL

- Guarda el estado propio del módulo.
- Los cambios de estructura se realizan con migraciones versionadas, nunca editando tablas a mano.
- La conexión exige TLS. La regla temporal para servicios Azure debe reemplazarse por red privada antes de producción.

Secretos

- La contraseña de PostgreSQL y la clave de Event Grid viven en el estado remoto y en secretos de Container Apps.
- No deben aparecer en Git, capturas, documentación, logs ni mensajes del equipo.
- Las variables con nombres de secretos no son el secreto: el valor real se inyecta en Azure.

Event Grid

- El backend tiene endpoint y clave para publicar CloudEvents 1.0.
- El tópico fue probado con devops.infrastructure.smoke-test y respondió HTTP 200.
- No hay suscripciones porque faltan endpoints, filtros y contratos confirmados de los módulos consumidores.

No confundir

Un evento anuncia un cambio de estado; no reemplaza la base de datos ni una llamada sincrónica que necesita respuesta inmediata.

ANTES, DURANTE Y DESPUÉS DEL DEPLOY

10. Checklist operativo de DevOps

Tarea	Estado comprobado	Evidencia
Antes	HECHO	CI verde, commit correcto, imagen SHA disponible, variables requeridas conocidas y plan sin bajas inesperadas.
Plan	HECHO	Revisar altas/cambios/bajas, recursos afectados, secretos sensibles ocultos y ambiente correcto.
Apply	HECHO	Autorizar solo el plan revisado, evitar despliegues simultáneos y observar la ejecución completa.
Smoke tests	HECHO	Frontend HTTP 200, /health, backend /api/health directo y por proxy, database: up.
Mensajería	HECHO	Estado del tópico Succeeded y publicación técnica aceptada. No usar un evento de dominio ficticio.
Convergencia	HECHO	Ejecutar plan posterior. El resultado esperado es No changes.
Evidencia	HECHO	Guardar enlace del workflow, SHA, plan, resultado de pruebas y decisión de rollback.

Rollback

Volver a una imagen anterior por digest/SHA cuando el problema es de aplicación. Si hay una migración incompatible, seguir el procedimiento específico de base de datos; nunca improvisar un rollback destructivo.

PENDIENTES SIN MAQUILLAR

11. Qué falta realmente

Tarea	Estado comprobado	Evidencia
Endpoints funcionales	PENDIENTE	El backend solo demuestra salud y conexión técnica; falta implementar el dominio de Obras Públicas.
Modelo y migraciones	PENDIENTE	Definir esquema, herramienta de migraciones y estrategia de evolución.
Suscripciones Event Grid	PENDIENTE	Esperar confirmación de contratos, consumidores, filtros y endpoints.
Release en test	PENDIENTE	Crear la primera release/vX.Y.Z y desplegarla mediante el workflow preparado.
Red privada	PENDIENTE	Reemplazar accesos públicos temporales antes de producción.
Presupuesto y alertas	PENDIENTE	Definir umbrales del crédito Azure for Students.
Producción	PENDIENTE	No está autorizada ni diseñada en este incremento.
PR del frontend original	PENDIENTE	El propietario debe aceptar el PR; development usa temporalmente el fork validado.

Conclusión

Tu parte DevOps de estas cuatro tarjetas está hecha. Lo pendiente ya no es provisionar la plataforma inicial: es completar la aplicación, acordar contratos y ejecutar el primer ciclo release -> test.

REFERENCIA RÁPIDA

12. Glosario, URLs y evidencia

PR

Propuesta revisable para incorporar cambios a otra rama.

SHA

Identificador único de un commit. Se usa como versión de la imagen.

Digest

Identidad inmutable del contenido exacto de una imagen.

Health check

Endpoint técnico que confirma que un servicio puede atender tráfico.

Smoke test

Prueba corta posterior al deploy para detectar fallos básicos.

Drift

Diferencia entre Terraform y lo que existe realmente en Azure.

Accesos de development

Frontend: [ca-obras-publicas-dev-frontend...azurecontainerapps.io](#)

Backend health: [ca-obras-publicas-dev-backend.../api/health](#)

Evidencia versionada

PR de Event Grid: [github.com/tomasgramajoDev/TPO/pull/10](#)

Deploy verificado: [GitHub Actions run 33501386215](#)

Documentación oficial consultada

Comparación de mensajería: [Microsoft Learn - Event Grid, Event Hubs y Service Bus](#)

Tópicos personalizados: [Microsoft Learn - Azure Event Grid custom topics](#)

Regla final

El código de infraestructura explica qué debe existir. El pipeline demuestra qué se intentó desplegar. Los smoke tests demuestran qué respondió. Se necesitan las tres evidencias para decir que un despliegue está terminado.